

FIRST-CLASS REFINEMENT TYPES IN SCALA



Matt Bovel, Viktor Kunčak and Martin Odersky

EPFL (Swiss Federal Institute of Technology in Lausanne, Switzerland)

October 4, 2026



INTRODUCTION

Slides and paper:



matt.bovel.net/scala-refinement-types/

Refinement types are types qualified with logical predicates.

For example,

$$\{x : \text{Int} \mid x > 0\}$$

denotes the type of all integers x such that $x > 0$.

In other languages: [Liquid Haskell](#),
[Boolean refinement types in F*](#),
[Subset types in Dafny](#), etc.

This talk presents:

1. A **prototype implementation** of refinement types in Scala 3 as *first-class* types (§2, §4).
2. A core **calculus** proven sound in Rocq by semantic typing, with subtyping and bounded polymorphism, under a *partial-correctness* semantics (§2.3, §3).

PART 1

IMPLEMENTATION



SYNTAX (§2.1)

Long form, mirroring set-builder notation:

```
type NonEmpty[A] =  
  {l: List[A] with l.nonEmpty}
```

Short form, reusing a name already in scope:

```
val x: (Int with x % 2 == 0) = 42  
// desugars to:  
val x: {v: Int with v % 2 == 0} = 42
```

```
def zip[A, B] (  
  xs: List[A],  
  ys: List[B] with ys.size == xs.size  
) : {zs: List[(A, B)] with zs.size == xs.size}
```

```
def concat[T] (  
  xs: List[T],  
  ys: List[T]  
) : {zs: List[T] with zs.size ==  
      xs.size + ys.size}
```

```
val xs: List[Int] = ...; val ys: List[Int] = ...  
zip(concat(xs, ys), concat(ys, xs))  
zip(concat(xs, ys), concat(xs, xs)) // error
```

WHY FIRST-CLASS? (§1)

Liquid Haskell is a plugin that runs **after** type checking. The type of `x` is declared twice:

```
{-@ x :: {v:Int | v mod 2 == 0} @-}  
let x = 42 :: Int in ...
```

“It’s sort of like you’re doing two things at once [...] you’re also talking to GHC, but you’re also talking to LiquidHaskell.” – [Usability Barriers for Liquid Types](#) (Gamboa et al., PLDI 2025)

Instead, we implement refinement as ordinary Scala types, participating in subtyping, inference, overloading and pattern matching.

```
// Example: Bounded polymorphism  
type Even = {v: Int with v % 2 == 0}  
def maximum[T: Ordering, U <: T]  
  (xs: List[U]): U = xs.reduce(max)  
def example1: Even =  
  maximum(List(2, 4, 6))
```

```
// Example: Overload resolution  
def min(l: List[Int] with l.isSorted) =  
  l.head // O(1)  
def min(l: List[Int]) = l.min // O(n)  
  
def ex2(l: List[Int] with l.isSorted) =  
  min(l) // calls the first overload
```

MIXED-PRECISION TYPE INFERENCE (§2.4)

Refinement types are **not** inferred from terms by default; the wider type is:

```
val x = 42 // Int, not {v: Int with v == 42}
```

Why not always infer the precise type?

1. **Backward compatibility:** implicit and overload resolution depend on inferred types; a more precise type changes which instances are found.
2. **Performance:** bigger types, slower comparisons.
3. **Usability:** inferring most precise types everywhere would be unreadable.

Instead, precision is recovered **on demand**, by a standard mechanism called *selfification*, triggered by the bidirectional type-inference algorithm:

```
val x: (Int with x == 42) = 42 // selfified
```

RUN-TIME CHECKS (§2.5)

Pattern matching branches on whether the predicate holds at run time:

```
type ID = {s: String with s.matches(idRegex)}
```

```
"a2e7-e89b" match
```

```
  case id: ID => ... // holds, id has type ID
```

```
  case _      => ... // does not hold
```

Checked casts. `.runtimeChecked` (SIP-57), when you expect the check to pass:

```
val id: ID = "a2e7-e89b".runtimeChecked
```

```
// desugars to:
```

```
val id: ID =
```

```
  if ("a2e7-e89b".matches(idRegex))
```

```
    "a2e7-e89b".asInstanceOf[ID]
```

```
  else throw new IllegalArgumentException()
```

SOLVER (§4.3)

How does the compiler check

`{x: T with p(x)} <: {y: S with q(y)} ?`

1. Check `T <: S`

2. Check `p(x)` implies `q(x)` for all `x`

Existing implementations lower to SMT.

Due to packaging and performance concerns, we instead implemented our own **lightweight e-graph-based solver**.

```
{v: Int with v == a && a == b}
  <: {v: Int with v == b}
{v: Int with a == b}
  <: {v: Int with f(a) == f(b)}
```

And domain-specific normalizations such as:

```
{v: Int with v == x + 3 * y}
  <: {v: Int with v == 2 * y + (x + y)}
```

Also beta-reduction, ADT constructors and limited reasoning for linear integer arithmetic.

EVALUATION (§4.4)

Compilation overhead, against each system's own unchecked baseline:

System	Overhead
First-class (ours)	0–12%
Schmid and Kunčák	20–38%
Stainless	$\geq 56\%$

When the feature is unused, the full Dotty CI passes unchanged: the compiler itself ($\approx 200\,000$ LoC), $\approx 10\,000$ tests, a 50-project community build.

$\leq 4\%$ slowdown on the official benchmark suite.

Expressiveness. Because refinements are types, we get type-argument inference. We also compile unmodified Scala, so adoption is incremental.

Conversely, both alternatives have stronger solvers: SMT-backed and complete for linear integer arithmetic, ADTs and equality.

PART 2

METATHEORY



LANGUAGE (§3.1)

Essentially System $F_{<:>}$ with refinements, dependent functions and pairs, sums, unions, intersections and equi-recursive types:

$$\begin{aligned} A, B ::= & X \mid \mathbf{Unit} \mid \mathbf{True} \mid \mathbf{False} \mid \mathbf{Int32} \mid \top \mid \perp \\ & \mid \Pi x:A. B \mid \forall(X:>L<:U). A \mid \Sigma x:A. B \\ & \mid A + B \mid \{x : A \mid p\} \mid A \vee B \mid A \wedge B \\ & \mid \mu X. A \end{aligned}$$
$$\begin{aligned} a, b, f, p ::= & c \mid x \mid \lambda x:A. b \mid f \ a \mid f \ [A] \\ & \mid \Lambda(X:>L<:U). b \mid \mathbf{let} \ x:A = a \ \mathbf{in} \ b \\ & \mid (a_1, a_2) \mid \mathbf{inl}[A] \ a \mid \mathbf{inr}[A] \ a \mid \mathbf{match} \dots \\ & \mid a \ \mathbf{op} \ b \mid \mathbf{if} \ a \ \mathbf{then} \ b_1 \ \mathbf{else} \ b_2 \mid \mathbf{loop}(a) \ x. b \end{aligned}$$
$$v ::= c \mid (v_1, v_2) \mid \mathbf{inl}(v) \mid \mathbf{inr}(v) \mid \langle \rho, \lambda x. b \rangle \mid \langle \rho, \Lambda X. b \rangle$$

Loops. $\mathbf{loop}(a) \ x. b$ is a limited recursion that suffices to model loops and tail recursion. The body returns $\mathbf{inl}$ to continue, $\mathbf{inr}$ to exit.

Contribution. To our knowledge, this is the first mechanized soundness proof combining refinements with $\vee/\wedge$, bounded polymorphism (both bounds), and positive equi-recursive types.

DEFINITIONAL INTERPRETER (§3.2)

Operational semantic is defined using a **fuel-bounded definitional interpreter**:

```
Fixpoint eval (fuel: nat) (env: list Value) (t: Term) : option (option Value) :=
  match fuel with
  | 0 => None (* timeout *)
  | S fuel' =>
    match t with
    | tabs _ b =>
      Some (Some (vabs env b))
    | tapp f a =>
      match eval fuel' env f with
      | Some (Some (vabs envf b)) =>
        ... eval fuel' (va :: envf) b
      | None => None (* timeout *)
      | _ => Some None (* stuck *)
    ...
```

INTERPRETATION (§3.3)

The **value interpretation** $\mathcal{V}\llbracket A \rrbracket_\delta^\rho(v)$ defines what it means for a value v to satisfy a type A , given a semantic type context δ and a value environment ρ .

$\mathcal{V}\llbracket A \rrbracket_\delta^\rho$ is a predicate `Value -> Prop`, also known as a **semantic type**.

The **term interpretation** $\mathcal{E}\llbracket A \rrbracket_\delta^\rho(t)$ lifts it to terms:

$$\mathcal{E}\llbracket A \rrbracket_\delta^\rho(a) \triangleq \forall n, r. \text{eval } n \ \rho \ a = \text{Some } r \implies \exists v. r = \text{Some } v \wedge \mathcal{V}\llbracket A \rrbracket_\delta^\rho(v)$$

“If evaluation terminates, it produces a value (not stuck), and that value is in $\mathcal{V}\llbracket A \rrbracket$.” Vacuously true for diverging terms; this is *partial correctness*.

Value interpretation of **function types**:

$$\mathcal{V}\llbracket \Pi x:A. B \rrbracket_\delta^\rho(v) \triangleq \exists \rho_f, b. v = \langle \rho_f, \lambda x. b \rangle \wedge \forall v_a. \mathcal{V}\llbracket A \rrbracket_\delta^\rho(v_a) \implies \mathcal{E}\llbracket B \rrbracket_\delta^{\rho_f[x \mapsto v_a]}(b)$$

Value interpretation of **refinement types**:

$$\mathcal{V}\llbracket \{x : A \mid p\} \rrbracket_\delta^\rho(v) \triangleq \mathcal{V}\llbracket A \rrbracket_\delta^\rho(v) \wedge \mathcal{E}\llbracket \text{True} \rrbracket_\delta^{\rho[x \mapsto v]}(p)$$

Value interpretation of **recursive types**:

$$\mathcal{V}\llbracket \mu X. A \rrbracket_\delta^\rho(v) \triangleq \forall n. F^n(v)$$

$$F^0(v) = \top, \quad F^{n+1}(v) = \mathcal{V}\llbracket A \rrbracket_{\delta[X \mapsto F^n]}^\rho(v)$$

TYPING (§3.4)

Semantic typing is **defined**. It quantifies over every well-formed context:

$$\Gamma \models a : A \triangleq \forall \delta, \rho. \text{wf}(\delta, \Gamma, \rho) \implies \mathcal{E}[\![A]\!]_{\delta}^{\rho}(a)$$

A context Γ is made of:

1. Term bindings: $x : A$
2. Type variable bindings: $X >: A <: B$
3. Equality facts: $s \sim t$

The usual typing rules are not definitions but **lemmas**: each rule is proven individually.

Rule for let-bindings:

$$\frac{\Gamma \models a : A \quad \Gamma, x : A, x \sim a \models b : B}{\Gamma \models \text{let } x:A = a \text{ in } b : \text{avoid}(B, x)} \text{ (T-Let)}$$

Selfification rule:

$$\frac{\Gamma \models a : A \quad \text{firstorder}(A)}{\Gamma \models a : \{x : A \mid x == a\}} \text{ (T-Self)}$$

SUBTYPING (§3.5)

Subtyping is semantic inclusion, again quantified over all well-formed environments:

$$\Gamma \models A <: B \triangleq \forall \delta, \rho. \text{wf}(\delta, \Gamma, \rho) \implies \mathcal{V}[[A]]_\delta^\rho \subseteq \mathcal{V}[[B]]_\delta^\rho$$

Every value satisfying A also satisfies B .

Recursive types are equi-recursive:

$$\frac{\text{spos}(X, A)}{\Gamma \models \mu X. A <: A[X \mapsto \mu X. A]} \text{ (S-Mu-Unfold)}$$

$$\frac{\text{spos}(X, A)}{\Gamma \models A[X \mapsto \mu X. A] <: \mu X. A} \text{ (S-Mu-Fold)}$$

Together they give $\mu X. A <:> A[X \mapsto \mu X. A]$, provided X occurs only **strictly positively** in A : never left of an arrow, nor in a $\forall$ bound.

REFINEMENTS SUBTYPING AND SEMANTIC IMPLICATION (§3.5)

Last but not the least, rules for refinement types:

$$\Gamma \models \{x : A \mid p\} <: A \text{ (S-RefineBase)}$$

Subtyping between refinements is semantic implication, a.k.a. entailment:

$$\frac{\Gamma \models A <: B \quad \Gamma, x : A \models p_1 \Rightarrow p_2}{\Gamma \models \{x : A \mid p_1\} <: \{x : B \mid p_2\}} \text{ (S-Refine)}$$

A refinement $\{x : A \mid p\}$ holds when p evaluates to `true` whenever it terminates. Implication reads both predicates that way:

$$\Gamma \models p_1 \Rightarrow p_2 \triangleq \forall \delta, \rho. \text{wf}(\delta, \Gamma, \rho) \implies \mathcal{E}[\mathbf{True}](p_1) \implies \mathcal{E}[\mathbf{True}](p_2)$$

Lemma:

$$\Gamma \models \{x : A \mid \text{diverge}\} <:> \{x : A \mid \mathbf{true}\} <:> A$$

FUTURE WORK

Implementation:

- **Better Solver** for what our lightweight solver cannot do.
- **Term-parameterized types**, to modularize predicates:

```
type Range(from: Int, to: Int) =  
  {v: Int with v >= from && v < to}
```
- **A termination checker**, needed only for the termination-sensitive entailment rules, never systematically for every function in a predicate.

Theory:

- **Rules for semantic implication.** We leave implication purely semantic; the solver's normalization rules are not yet verified against it.
- **Enforcing purity.** Today a function called in a predicate is *assumed* pure. Capture and separation checking can ensure that predicates are pure.
- **Classes and objects**, absent from the presented core calculus.

CONCLUSION

We showed:

1. A **prototype implementation** of refinement types in Scala 3 as *first-class* types; normal Scala types that participate in subtyping, inference, overloading and pattern matching.
2. A core **calculus** proven sound in Rocq by semantic typing. It includes refinements, dependent functions and pairs, sums, unions, intersections and equi-recursive positive types, and allows predicates to diverge.

Slides and paper:



matt.bovel.net/scala-refinement-types/



Un type raffiné,
by Marina Granados Castro

BACKUP: LH USABILITY BARRIERS

From “Usability Barriers for Liquid Types” [1]:

- 4.2 Unclear Divide between Haskell and LiquidHaskell:
 - “comments are usually seen as just optional information in the code and not something that is directly used by the compiler”
 - “It’s sort of like you’re doing two things at once because you’re implementing in Haskell. But you’re also talking to GHC, but you’re also talking to LiquidHaskell.”
- 4.7 Unhelpful Error Messages
 - “[...] error messages produced from typing errors inside the predicates, seemed indistinguishable from those produced by verification errors.”
- 4.8 Limited IDE Support
 - “[user] tried to use the function `length`, but since it was not imported, it was impossible to use in this case.”

[1] Catarina Gamboa, Abigail Reese, Alcides Fonseca, and Jonathan Aldrich. 2025. Usability Barriers for Liquid Types. Proc. ACM Program. Lang. 9, PLDI, Article 224 (June 2025), 26 pages. [doi:10.1145/3729327](https://doi.org/10.1145/3729327)

BACKUP: `List.collect`

Scala type parameters are *erased* at runtime, so we cannot match on a `List[T]`.

However, we can use `.collect` to filter and convert a list:

```
type Pos = { v: Int with v >= 0 }
```

```
val xs = List(-1, 2, -2, 1)
```

```
xs.collect { case x: Pos => x } : List[Pos]
```

BACKUP: SPECIFY USING ASSERTIONS 🙄

We can use assertions:

```
def zip[A, B](  
  xs: List[A],  
  ys: List[B]  
) : List[(A, B)] = {  
  require(xs.size == ys.size)  
  ...  
}.ensuring(_.size == xs.size)
```

Limitations:

- *Runtime overhead*: checked at runtime, not compile time,
- *No static guarantees*: only checked for specific inputs,
- *Not part of the API*: not visible in function type,
- *Hard to compose*: cannot be passed as type argument.

BACKUP: SPECIFY USING DEPENDENT TYPES 🙄

Can we use path-dependent types?

```
def zip[A, B] (  
  xs: List[A],  
  ys: List[B] {  
    val size: xs.size.type  
  }  
) : List[(A, B)] {  
  val size: xs.size.type  
} = ...
```

Limitations:

- *Limited reasoning*: only fields, literals and constant folding,
- *Not inferred*: need manual type annotations, or not typable at all,
- *Different languages*: term-level vs type-level.

FUTURE WORK: TERM-PARAMETERIZED TYPES

```
extension [T](list: List[T])  
  def get(index: Int with index >= 0 && index < list.size): T = ...
```

To modularize the “range” concept, we could introduce term-parameterized types:

```
type Range(from: Int, to: Int) = {v: Int with v >= from && v < to}  
extension [T](list: List[T])  
  def get(index: Range(0, list.size)): T = ...
```